

PYTHON MODULE 02

Sure! Here's a **detailed but simple explanation** of the **Object Oriented Programming (OOP)** topics you've mentioned, written in an **exam-like but easy-to-understand** manner. Each topic is explained clearly with sub-topics and key properties where needed.

▣ Object Oriented Programming (OOP)

Object Oriented Programming is a programming approach based on the concept of "objects". It helps organize complex programs using **classes** and **objects**, making them reusable, scalable, and easier to manage.

📖 1. Classes and Objects

✓ Class:

A **class** is like a blueprint or template to create objects. It defines attributes (data) and methods (functions) that describe the behavior and characteristics of the object.

class Car:

```
    color = "Red" # Attribute
```

```
    def start(self): # Method
```

```
        print("Car started")
```

✓ Object:

An **object** is an instance of a class. When a class is defined, no memory is allocated until we create an object.

```
my_car = Car() # my_car is an object of class Car
my_car.start() # Output: Car started
```

📖 2. Class Attributes

Class attributes are variables that are **shared across all objects** of the class. They are defined inside the class but **outside any method**.

✓ Properties:

- Shared by all instances.
- Can be accessed using class name or object.

class Student:

```
    school_name = "ABC School" # Class attribute
print(Student.school_name) # Access via class
s1 = Student()
print(s1.school_name)      # Access via object
```

📖 3. Methods in Class

✓ a. Regular Methods:

Defined using def, take self as the first parameter.

```
class Dog:
    def bark(self): # Method with self
        print("Woof!")
```

✓ b. Private Methods:

Private methods **are meant to be used only inside the class.**

We declare them by using a double underscore prefix `__`.

```
class Secret:
    def __private_method(self):
        print("This is private")
```

```
def access(self):
    self.__private_method() # Accessible within class
```

 **Cannot be accessed directly like `obj.__private_method()`**

4. Constructor

A **constructor** is a **special method called automatically when an object is created.** It's used to initialize the object's attributes.

✓ **In Python, the constructor is: `__init__()`**

```
class Person:
    def __init__(self, name):
        self.name = name # instance attribute
```

```
p1 = Person("Conan")  
print(p1.name) # Output: Conan
```

✓ Properties:

- Automatically called.
 - Used to set default values during object creation.
-

📖 5. Destructor

A destructor is used to clean up resources (like closing a file) when an object is deleted.

✓ In Python, the destructor is: `__del__()`

class File:

```
def __del__(self):  
    print("Destructor called, object deleted")
```

```
f1 = File()
```

```
del f1 # Destructor called
```

📖 6. Built-in Methods in Class

Python provides many **built-in methods** for classes that start and end with double underscores (also called **magic methods** or **dunder methods**).

✓ Examples:

- ✓ • `__init__()` – Constructor
- ✓ • `__del__()` – Destructor
- ✓ • `__str__()` – String representation of object
- ✓ • `__len__()` – Returns length (if applicable)
- ✓ • `__repr__()` – Developer-friendly string of object

class Book:

```
def __str__(self):  
    return "Book object"
```

b = Book()

print(b) # Output: Book object

📖 7. Built-in Attributes in Objects

Python automatically assigns some built-in attributes to every object.

✓ Common built-in attributes:

- ✓ • `__dict__` – Returns a dictionary containing the object's namespace.
- ✓ • `__class__` – Tells the class to which an object belongs.
- ✓ • `__doc__` – Returns the docstring of the class.

- ✓ __module__ – Returns the name of the module in which the object is defined.
- ✓ __name__ – Returns the name of the class/function/module.

```
class Animal:
```

```
    """This is an animal class"""
```

```
    pass
```

```
a = Animal()
```

```
print(a.__class__) # Output: <class '__main__.Animal'>
```

```
print(a.__doc__) # Output: This is an animal class
```

Summary Table

Concept	Description
Class	Blueprint/template for objects
Object	Instance of a class
Class Attribute	Shared variable for all objects
Method	Function defined in a class
Private Method	Method with double underscore, used only within class
Constructor (<u>__init__</u>)	Initializes object, called automatically

Concept	Description
Destructor (<code>__del__</code>)	Cleans up resources, called during object deletion
Built-in Methods	Special methods like <code>__str__</code> , <code>__len__</code> , <code>__repr__</code> etc.
Built-in Attributes	Special properties like <code>__class__</code> , <code>__dict__</code> , <code>__doc__</code> etc.

Would you like a PDF version of this explanation too, for printing or revision?